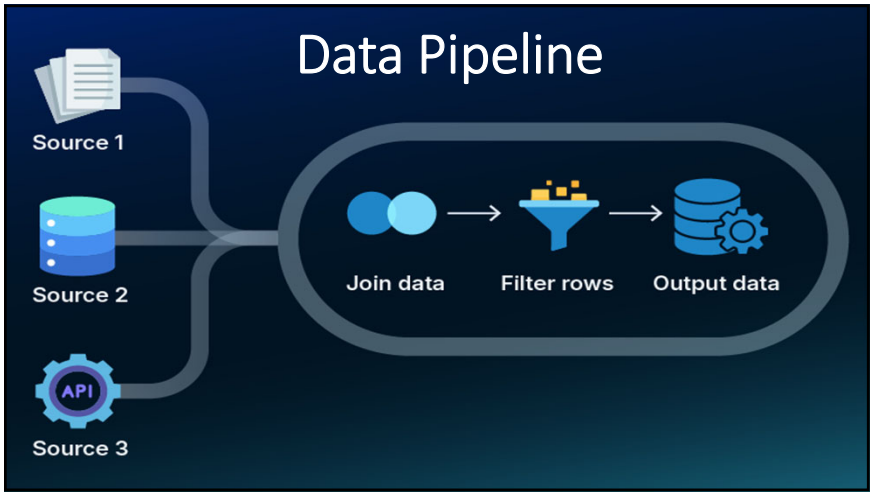
ENGE707 — WEEK 3

Cleaning Real Data Before You Trust It

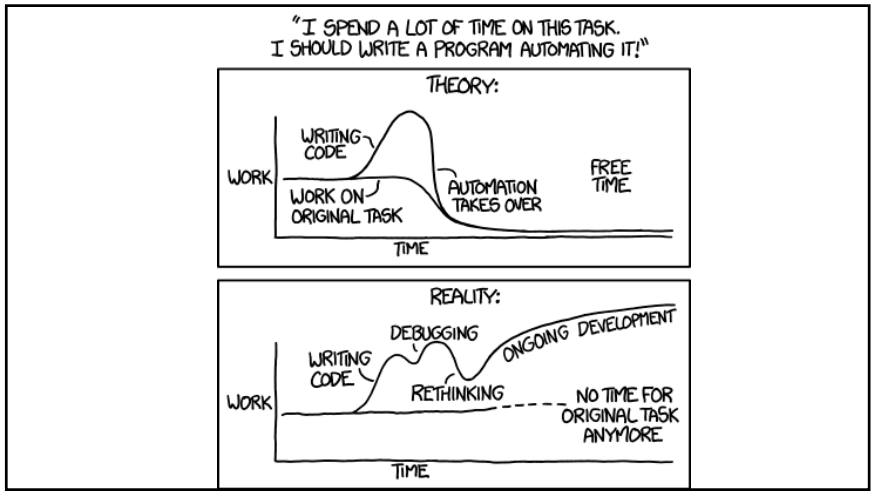
Loading a real medical dataset, and building a pipeline that survives contact with messy reality

ENGE707 • Data & AI Foundations

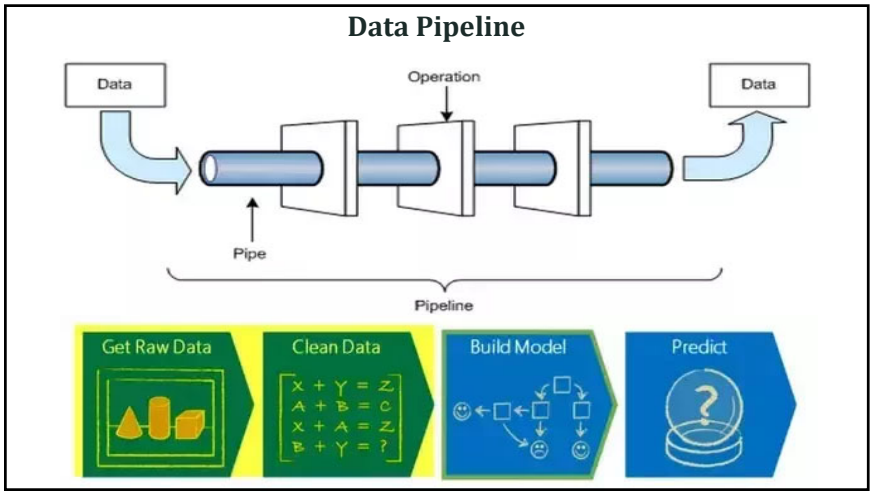
1



2



3



4

What is your data pipeline/workflow?

Data pipelines are sequences of processing and analysis steps applied to data for a specific purpose.

They're useful in production projects, and they can also be useful if one expects to encounter the same type of business question in the future, so as to save on design time and coding.

For instance, one could remove outliers, apply dimensionality reduction techniques, and then run models to provide automatic classification on a particular dataset that is pulled every week.

What is your data pipeline/workflow?

A data scientist constantly tries new ideas and changes steps of their pipeline, it is not possible to complete everything in one pass - it is what makes DS job different:

- *extract new features and accidentally find noise in the data*
- *clean up the noise, find one more promising feature*
- *extract the new feature*
- *rebuild and validate the model, realize that the learning algorithm parameters are not perfect for the new feature set*

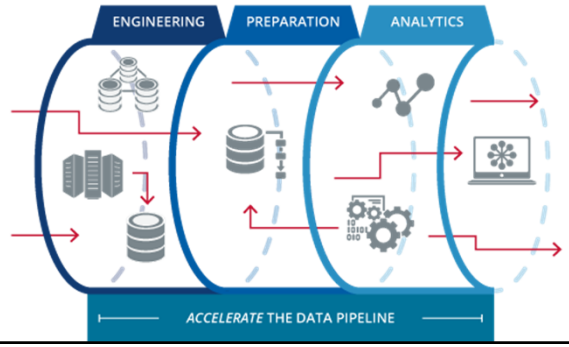
What is your data pipeline/workflow?

A data scientist constantly tries new ideas and changes steps of their pipeline, it is not possible to complete everything in one pass - it is what makes DS job different:

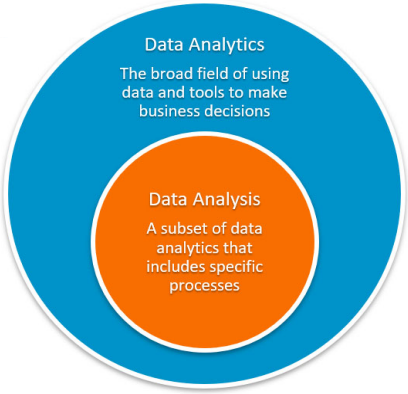
- *change machine learning algorithm parameters and retrain the model*
- *find the ineffective feature subset and remove it from the feature set*
- *try a few more new features*
- *try another ML algorithm. And then a data format change is required*

What is your data pipeline/workflow?

A data pipeline is a set of actions that extract data (or directly analytics and visualization) from various sources.



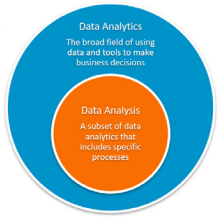
Data Analysis vs Data Analytics



9

Data Analysis vs Data Analytics

- Data analysis involves examining, cleaning, transforming, and modelling data to uncover meaningful insights, patterns, and trends. It focuses on extracting actionable information from datasets to support decision-making processes.
- Data analytics encompasses a broader range of activities, including data collection, processing, analysis, interpretation, and visualization. It aims to generate insights, inform strategies, and drive business outcomes through data-driven approaches.



10

Data Cleaning

- Data cleaning, also known as data cleansing or data scrubbing, refers to the process of detecting and correcting errors, inconsistencies, and inaccuracies in a dataset to improve its quality and reliability for analysis.
- Clean data is essential for accurate and meaningful analysis, as errors and inconsistencies can lead to biased results and erroneous conclusions.

11

Data Cleaning

- Process of identifying and correcting errors in datasets
- Focuses on preparing data for analysis
- Common tasks:
  - Remove duplicate records
  - Correct missing or invalid values
  - Standardize formats (dates, text, units)
  - Fix inconsistencies and typographical errors

12

Data Cleansing

- Broader process of improving overall data quality
- Includes data cleaning plus business rule validation
- May involve:
  - Data validation
  - Data standardization
  - Data enrichment
  - Ongoing monitoring and quality management

13

05

Descriptive Statistics

Numerical summaries — before a single chart is drawn

14

WHY SUMMARISE FIRST

Five numbers, one purpose

Mean

- the average — best when the distribution is reasonably symmetric.

Median

- the middle value once sorted — barely moves even if a handful of extreme values are added or removed.

Mode

- the single most common value.

Standard deviation

- how far, typically, values sit from the mean.

Skew

- which direction the long tail points — 0 is symmetric, positive is a right tail, negative is a left tail.

15

DESCRIPTIVE STATS, PIECE 1 OF 3

The describe() table, in full detail

age column, all 448 values

age
45
62
...

→

.describe()

mean	50%	max
47.2	47.0	89

8 stats, one call

```
print(df_etl_clean[["age", "height", "weight", "heart_rate"]].describe())
```

WHAT'S HAPPENING

Selecting four columns with double brackets — a DataFrame, not a Series.

- This is the exact Series-vs-DataFrame distinction from earlier, put to use: describe() on a DataFrame runs once and returns a full table, one row of statistics per column.

The output includes count, mean, std, min, 25%, 50%, 75%, max — eight rows, automatically.

- The 50% row is the median — describe() never prints the word "median" directly, worth knowing so you don't go looking for a row that isn't labelled that way.

count is worth checking first, not last.

- If count is lower than 448 for any column, that column still has missing values describe() is silently excluding — a real, easy-to-miss signal.

16

DESCRIPTIVE STATS, PIECE 2 OF 3

Computing skew for each column, one at a time

weight: 65, 70, 68, 190, 72

weight
65, 70, 68...
190 (rare, high)

→ .skew().round(2)

age	weight
-0.01	1.61

one column per loop

```
for col in ["age", "height", "weight", "heart_rate"]:
    print(col, df_etl_clean[col].skew().round(2))
```

WHAT'S HAPPENING

Why a loop here, and not a single .skew() call on all four at once?

- df_etl_clean[["age",...]].skew() would actually work too and return all four at once — the loop is used here purely so each result prints with its own column name label, easier to read line by line while explaining it live.

.round(2)

- keeps the printed number to 2 decimal places — skew doesn't need more precision than that to be useful.

What the actual numbers mean once printed:

- a value near 0 — roughly symmetric. A larger positive number — a real right tail (like weight). A larger negative number — a real left tail (like height, once the impossible values are gone).

17

NUMBERS, NOT JUST CODE

Worked example: reading skew, by hand

WORKED EXAMPLE — 10 heart rate readings

```
HR = [72, 75, 68, 80, 74, 71, 77, 73, 76, 70]
```

Step 1 — mean:

```
(72+75+68+80+74+71+77+73+76+70) / 10 = 736 / 10 = 73.6
```

Step 2 — sort the values:

```
68, 70, 71, 72, 73, 74, 75, 76, 77, 80
```

Step 3 — compare mean to the middle value:

```
middle (median) ≈ 73.5.  Mean (73.6) is almost identical to the median.
```

Step 4 — interpretation:

skew ≈ 0 — roughly symmetric.

```
If one more reading of 140 were added, the mean would jump but the median barely would —
that gap growing is exactly what a positive skew number captures.
```


18

DESCRIPTIVE STATS, PIECE 3 OF 3

The target variable's own balance

448 rows, raw counts

False	True
244	204

→ normalize=True

False	True
0.545	0.455

count → proportion

```
print(df_etl_clean["arrhythmia_present"].value_counts(normalize=True))
```

WHAT'S HAPPENING

value_counts()

- counts how many rows fall into each unique value — here, just True and False.

normalize=True

- converts those raw counts into proportions that sum to 1, instead of raw counts — 0.55 reads more immediately as "55%" than a bare row count would.

Why check this at all?

- If one outcome were rare — say 95% normal, 5% arrhythmia — that imbalance would need to be accounted for later, especially once machine learning enters the picture. Checking it here, early, costs one line.

19

06

Histograms

Seeing the shape a statistics table can only imply

20

THE ANATOMY

Reading a histogram

Bars

- the number of patients whose value falls inside each range (“bin”) — not each exact value.

bins parameter

- controls how many of these ranges the data gets split into — too few bins hides real structure, too many makes the shape look noisy.

Peak

- the tallest bar — the range containing the most patients.

Isolated bar, far from the rest

- a possible invalid value or genuine outlier — exactly how the impossible 780cm height was first spotted.

21

HISTOGRAMS, PIECE 1 OF 4

A single histogram, built line by line

heart_rate: 72, 78, 65, 90…

heart_rate
448 values

→

bucketed into 20 bins

70-75 bpm	count
bar height	58

```
import matplotlib.pyplot as plt
plt.hist(df_etl_clean["heart_rate"].dropna(), bins=20)
plt.xlabel("Heart rate (bpm)")
plt.ylabel("Number of patients")
plt.title("Distribution of heart rate")
plt.show()
```

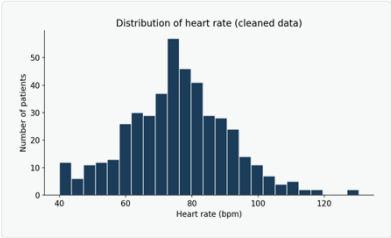
- **.dropna()**
- removes any remaining missing values right before plotting — plt.hist() cannot plot a gap, and will error without this.
- **bins=20**
- an explicit choice, not the default — chosen here to show enough shape detail without looking noisy for a dataset this size.
- **plt.xlabel / plt.ylabel / plt.title**
- three separate calls, each labelling one part of the same chart — all three needed, or the chart is unreadable without the code beside it.
- **plt.show()**
- the very last line — nothing after this point in the same code block affects this chart.

22

HISTOGRAMS, PIECE 2 OF 4 — REAL OUTPUT

What that exact code actually produces

- **This is the literal output of the code on the previous slide.**
- Not a separate illustration — same column, same bins=20, run on the cleaned data.
- **Roughly bell-shaped, one clear peak.**
- Consistent with heart rate's skew being the mildest of the four columns checked earlier.



23

HISTOGRAMS, PIECE 3 OF 4

Three columns, before any fixing

height: cm, weight: kg, HR: bpm

height	weight	HR
780	70	72

→ *subplots(1,3)*

3 separate panels

axes[0]	axes[1]	axes[2]
height	weight	HR

```
fig, axes = plt.subplots(1, 3, figsize=(12, 4))
axes[0].hist(df_original["height"].dropna(), bins=30)
axes[1].hist(df_original["weight"].dropna(), bins=30)
axes[2].hist(df_original["heart_rate"].dropna(), bins=30)
axes[0].set_title("Height"); axes[1].set_title("Weight"); axes[2].set_title("Heart rate")
plt.show()
```

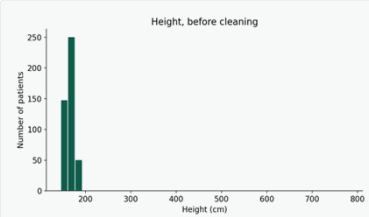
- **plt.subplots(1, 3, ...)**
- creates one figure containing 3 separate chart areas side by side, returned as fig and axes — axes is a list, one entry per chart area.
- **axes[0], axes[1], axes[2]**
- each one is plotted on separately — same .hist() method as before, just called on a specific chart area instead of on plt directly.
- **Why three separate calls, not one shared histogram?**
- height, weight, and heart rate are on completely different scales (cm, kg, bpm) — combined on one chart, the smaller-range column would be squashed flat and unreadable.
- **df_original, deliberately — not df_etl_clean.**
- This is the “before” view, on purpose — the impossible height value is still in this data.

24

HISTOGRAMS, PIECE 4 OF 4 — REAL COMPARISON

Before vs. after — the visual difference cleaning made

- **Left chart: height, straight from df_original.**
- The 780cm value sits as its own isolated bar, visually separated from every real patient.
- **After clean_data() runs, that same chart no longer has this bar at all.**
- Not capped, not hidden — the row was removed entirely by the invalid-value check from Section 05.
- **This single chart is the real justification for that removal.**
- Not an assumption stated in text — something you can see directly, before and after.



25

RECAP

Key takeaways

- **Read the documentation before writing a single line of code.**
- A wrong guess doesn't error — it silently gives you a wrong answer.
- **Invalid values and statistical outliers are not the same thing.**
- One gets corrected confidently; the other gets flagged and reviewed in context.
- **One cleaning function, every rule, built up piece by piece.**
- Auditable, reusable, and safe to re-run on a fresh copy of the raw data.
- **ETL vs ELT is a real trade-off, tested with real code today.**
- Stable rules → ETL. Rules likely to change → ELT.

26

07

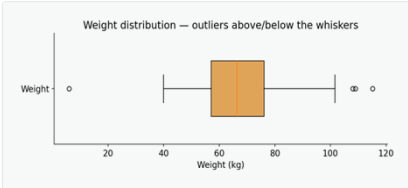
Boxplots & Outliers

Detecting statistical rarity — and knowing it isn't the same as an error

27

THE ANATOMY

How to read a boxplot



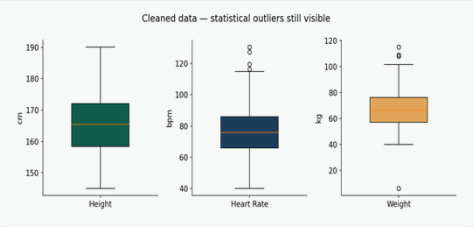
- **Box**
- The middle 50% of values, from Q1 to Q3.
- **Line inside the box**
- The median, not the mean.
- **Whiskers**
- Extend to the furthest value within 1.5 × IQR.
- **Dots beyond the whiskers**
- Statistical outliers — unusual, but not automatically wrong.

28

DETECT

Three columns, three different shapes

- **Different units need separate panels.**
- Combining columns with different scales would squash some plots flat.
- **Height, heart rate, and weight are checked with the IQR method.**
- Compare each flagged point with age, height, weight, and diagnosis context — never judge from the boxplot alone.



POTENTIAL CONCERNS

Real cases flagged — what happened to each

- **Age 3, height 105cm, weight 12kg —**
- very low BMI, but plausible for a young child. Kept.
- **Age 53, height 169cm, weight 176kg —**
- extremely high BMI, but a real, documented condition. Kept.
- **Two records, identical demographics, different diagnosis class —**
- checked in full before deciding; not exact duplicates, so both retained.

NUMBERS, NOT JUST CODE

Worked example: z-score, by hand

WORKED EXAMPLE — Same 10 heart rate readings

```
HR = [72, 75, 68, 80, 74, 71, 77, 73, 76, 70]      mean = 73.6
```

Step 1 — deviation of each value from the mean:

```
-1.6, 1.4, -5.6, 6.4, 0.4, -2.6, 3.4, -0.6, 2.4, -3.6
```

Step 2 — square each deviation, average them, then square-root:

```
standard deviation = √( sum of squared deviations / 9 ) = 3.57
```

Step 3 — z-score formula:

```
z = (value − mean) / standard deviation
```

Step 4 — for the highest reading, 80:

z = (80 − 73.6) / 3.57 = 1.79

A z of 1.79 means this value sits 1.79 standard deviations above the mean — unusual, but nowhere near the |z| > 3 rule of thumb some methods use for a strong outlier.

NUMBERS, NOT JUST CODE

Worked example: IQR bounds, by hand

WORKED EXAMPLE — Same 10 readings, sorted

```
68, 70, 71, 72, 73, 74, 75, 76, 77, 80
```

Step 1 — Q1 (25th percentile) and Q3 (75th percentile):

```
Q1 = 71.25      Q3 = 75.75
```

Step 2 — IQR:

```
IQR = Q3 − Q1 = 75.75 − 71.25 = 4.5
```

Step 3 — the whisker bounds:

```
lower bound = Q1 − 1.5×IQR = 71.25 − 6.75 = 64.5
upper bound = Q3 + 1.5×IQR = 75.75 + 6.75 = 82.5
```

Step 4 — check every value against both bounds:

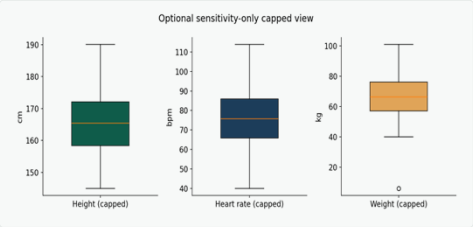
All 10 values fall between 64.5 and 82.5 — no outliers in this small sample.

A value of 90 would break the upper bound and get flagged; this sample simply doesn't contain one.

STEP 3 — SENSITIVITY CHECK

Optional: a sensitivity-only capped view

- **A separate copy, never the main dataset.**
- df_sensitivity — capped for comparison only. df_etl_clean itself is never touched.
- **Why keep this separate?**
- Shows whether conclusions depend strongly on extreme values, without silently rewriting the data everyone else uses.



33

08

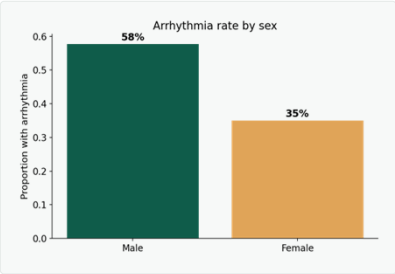
Bar Charts & Correlation

Comparing groups, and finding relationships between measurements

34

BAR CHART

Arrhythmia rate by sex



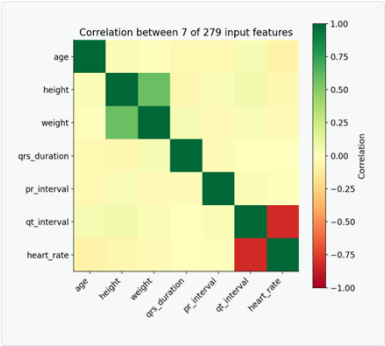
- **What each bar shows**
- The proportion of patients with any arrhythmia, within one sex category.
- **A real, visible gap.**
- One sex shows a noticeably higher proportion than the other.
- **Next step**
- The chi-square test later checks whether this visible difference is statistically significant.

35

7 OF 279 COLUMNS

Correlation heatmap

- **Near +1**
- the two variables increase together.
- **Near −1**
- one increases while the other decreases.
- **Near 0**
- little or no straight-line relationship.
- **Important**
- Correlation shows association, not cause — a high value never proves one variable causes the other.



36

? DISCUSS

Before the reveal: which pair do you predict correlates most strongly?

Pick two columns from the heatmap and guess the direction — positive or negative.

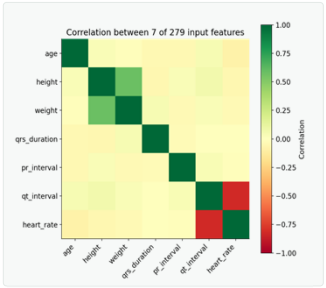
Turn to your neighbour — 60 seconds

37

Which pair correlates most strongly?

✓ ANSWER

QT interval and heart rate — the strongest relationship on the whole grid, and a negative one: as heart rate increases, QT interval tends to decrease. Height and weight are the second-strongest, and positive — taller patients generally weigh more. Most other pairs stay close to zero.



Both of these are real, physiologically expected relationships — not coincidences of this particular sample.

38

NUMBERS, NOT JUST CODE

Worked example: correlation direction, by hand

WORKED EXAMPLE — 5 patients — heart rate paired with QT interval

```
heart_rate:   70,  75,  80,  65,  85
qt_interval: 400, 390, 370, 410, 360
```

Step 1 — look at the direction, pair by pair:
As heart_rate rises (70→75→80→85), qt_interval consistently falls (400→390→370→360). Every single pair moves in opposite directions — no exceptions in this sample.

Step 2 — what the correlation formula does with this:
it measures how consistently two variables move together (or apart), scaled to sit between −1 and +1.

Step 3 — computed on these exact 5 pairs:

correlation = −0.99

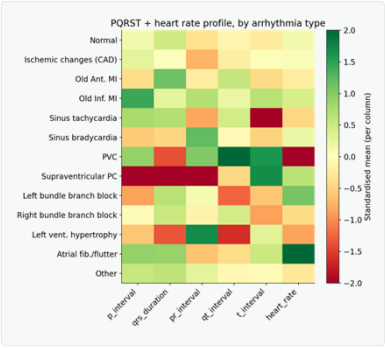
As close to a perfectly straight negative line as 5 points can get — which is exactly what "every pair moves in opposite directions, with no exceptions" looks like numerically.

39

13 REAL DIAGNOSIS TYPES

Pattern heatmap by arrhythmia type

- **Sinus tachycardia:**
- very high heart rate, lower QT interval.
- **Sinus bradycardia:**
- very low heart rate, higher QT interval.
- **Left bundle branch block:**
- strongly increased QRS duration — matches its clinical definition exactly.
- **Atrial fibrillation/flutter:**
- much lower P and PR interval — the loss of a normal P wave.



40

READING THIS CAREFULLY

Colours here are relative, not raw values

- **A dark-green cell does not mean the raw value is the largest.**
- It means that arrhythmia type sits well above the overall mean, for that one specific measurement.
- **These are exploratory group averages, not diagnostic rules.**
- Classes with only a handful of patients (PVC: 3, Supraventricular PC: 2) can produce unstable-looking patterns simply from small sample size.

41

09

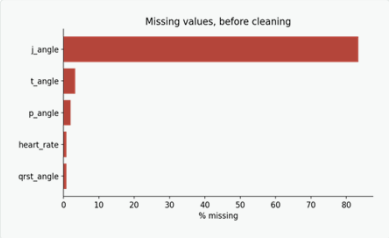
Missing Values & Scatter Plots

Finding gaps, and comparing individual patients directly

42

WHERE ARE THE GAPS?

The missing-value bar chart



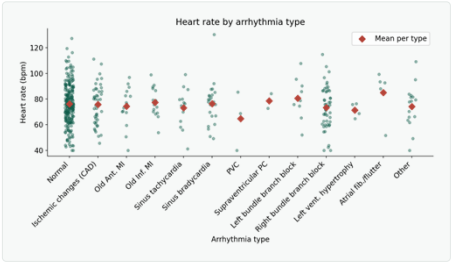
- **Longer bar = more missing.**
- j_angle should have the longest bar — most of its raw values are missing.
- **Use this chart to decide:**
- fill it, investigate it, or remove the column entirely.

43

THE USEFUL SCATTER PLOT

Heart rate by arrhythmia type

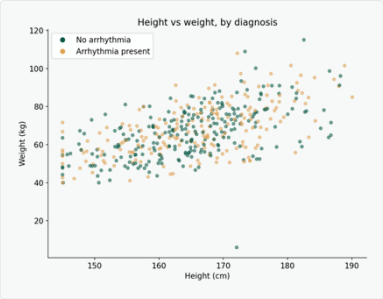
- **Each dot: one patient. Each red diamond: one type's mean.**
- Sinus tachycardia has the highest mean; sinus bradycardia the lowest — exactly matching their names.
- **The dots overlap heavily between most other groups.**
- Heart rate strongly separates tachycardia/bradycardia specifically, but is far less useful for telling the other categories apart.



44

A WEAKER COMPARISON

Height vs weight, by diagnosis



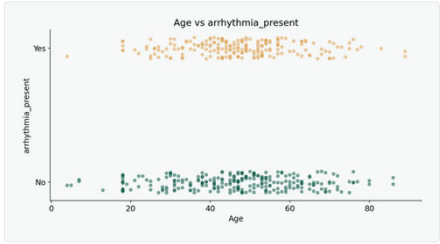
- **Height and weight relate to each other.**
- Taller patients generally weigh more — the diagonal lean is real.
- **Diagnosis colour does not separate cleanly.**
- Orange and teal overlap almost everywhere — these two variables alone don't distinguish arrhythmia from normal.

45

CONTINUOUS VS YES/NO

Age vs arrhythmia_present

- **Jitter, not new data.**
- A small random vertical nudge only, added purely so overlapping points become visible.
- **Both groups span almost the entire age range.**
- Age alone does not clearly separate arrhythmia from normal cases — no obvious cut-off exists.



46

?

DISCUSS

Which of the last three scatter plots was actually the most useful — and why?

Height/weight, heart rate/type, or age/diagnosis? Defend your choice.

Turn to your neighbour — 60 seconds

47

Which scatter plot was the most useful?

✓ ANSWER

Heart rate by arrhythmia type. It's the only one of the three where the group means are genuinely spread out, not clustered together — sinus tachycardia and sinus bradycardia sit at real, distinct extremes. Height vs weight and age vs diagnosis both show heavy overlap between groups, meaning neither variable, alone, tells the two outcomes apart.

Usefulness isn't about which chart type is fanciest — it's about whether the groups you're comparing actually separate on that specific variable.

48

10

Pair Plots

Scanning several relationships in one grid, instead of one chart at a time

49

PAIR PLOT

Four variables, one grid

- **Diagonal: each column's own distribution.**
- **Off-diagonal: every pairwise scatter.**
- height/weight and QT/heart_rate both show real structure — the rest stay flat clouds.
- **Diagnosis colour overlaps in nearly every panel.**
- These four variables alone don't cleanly separate arrhythmia from normal — classification would need more features.

50

11

Population vs Sample

Why every number we've computed is an estimate, not the truth

51

FOUNDATIONS FOR TESTING

Four terms, one core idea

Population

Every possible patient we want to understand — not just these 448.

Sample

The 448 patients actually included in this dataset.

Parameter

The true value in the full population — usually unknown.

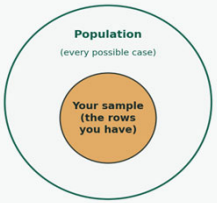
Statistic

A value calculated from the sample, used to estimate the parameter.

52

VISUALISING THE IDEA

Sample drawn from a population



Every statistic computed this week — every mean, every skew, every correlation — estimates a population value, never equals it exactly.

53

12

Hypothesis Testing

Turning a pattern you can see into a claim you can defend

54

CORE VOCABULARY

The testing decision path

55

KEY TERMS

The vocabulary you actually need

- **H0 (null hypothesis):**
- no real difference or relationship exists.
- **H1 (alternative):**
- a real difference or relationship does exist.
- **p-value:**
- the probability of seeing a result this extreme, if H0 were actually true.
- **α (usually 0.05):**
- the agreed cut-off for calling a result “statistically significant.”
- **Type I error:**
- rejecting a true H0 — a false alarm.
- **Type II error:**
- failing to reject a false H0 — a missed real effect.

56

?

DISCUSS

A test finds “no significant difference,” but you strongly suspected there was one. What went wrong?

- Maybe there genuinely is no real effect
- Maybe there IS a real effect, but the test missed it — a Type II error

Consider both possibilities before answering: is the effect real, or isn't it?

Turn to your neighbour — 60 seconds

57

A test finds “no significant difference” — what went wrong?

✓

ANSWER

Possibly nothing went wrong at all — the honest answer might simply be that there is no real effect to find, and your suspicion was mistaken. But it's also genuinely possible this is a Type II error: a real effect exists, but this specific sample, by chance, didn't show it clearly enough to cross the significance threshold. A non-significant result never proves H0 is true — it only means this test, on this data, didn't find enough evidence against it.

This is exactly the situation in the real t-test coming up next: a small, non-significant heart-rate difference that could genuinely be “no effect,” or could be a real effect this particular sample was too small to detect clearly.

58

13

The t-test

Comparing two group averages — heart rate by diagnosis

59

THE MECHANICS

What a t-test actually does

- **Compares the mean of a numeric column between two groups.**
- Asks whether the difference is large enough to be real, or small enough to be explained by which patients happened to end up in this sample.
- **The t-statistic:**
- how many standard errors apart the two means are. Larger t, relative to the natural spread — smaller p-value.
- **When to use it:**
- one numeric variable, exactly two groups. More than two groups, or comparing counts instead of averages, needs a different test.

60

NUMBERS, NOT JUST CODE

Worked example: the t-statistic, by hand

WORKED EXAMPLE — Two small groups (illustrative)

```
Arrhythmia present:  74, 76, 78     → mean = 76
Normal:              70, 72, 74     → mean = 72
```

Step 1 — the difference in means:

`76 − 72 = 4`

Step 2 — how spread out each group is on its own:

`both groups here have the same small internal spread — values 2 apart within each group.`

Step 3 — the t-statistic compares the two:

`t = difference in means ÷ how much variation is expected just from sampling — a bigger gap relative to that spread gives a bigger |t|.`

Step 4 — computed on these exact 6 numbers:

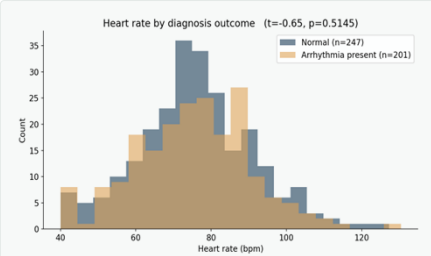
t = 2.45, p = 0.070

`p is just above 0.05 here — illustrating, with a tiny sample, exactly the "close but not quite significant" result the real heart-rate t-test also produced.`

THE REAL RESULT

Heart rate by diagnosis outcome

- **Question:**
- is average heart rate different depending on whether arrhythmia is present?
- **The two distributions overlap heavily.**
- Visually, this is already a clue about what the test will find.
- **Result: not statistically significant.**
- p > 0.05 → fail to reject H0. The sample difference is not large enough relative to the variation within each group.



WHAT THE NUMBERS MEAN

Reading the result correctly

- **A larger absolute t-value — stronger evidence of a real difference.**
- **A small p-value — the observed difference would be unusual if there were truly no effect.**
- **A large p-value — the observed difference could easily happen through ordinary sampling variation.**
- **"Not significant" is not "proven equal."**
- It means this test, on this sample, did not find strong enough evidence of a difference — a real, honest, useful result, not a failed analysis.

14

Chi-Square

Comparing category counts — sex vs diagnosis

THE MECHANICS

What a chi-square test actually does

- **Compares category counts, not averages.**
- Takes a cross-tab table and checks whether the actual counts are close to what you'd expect if the two variables were completely unrelated.
- **Bigger deviation from "expected if unrelated"**
- → larger chi-square statistic → smaller p-value.
- **When to use it:**
- both variables are categories (sex: M/F, diagnosis: present/absent) — not numbers to average, which is exactly why a t-test would not make sense here.

NUMBERS, NOT JUST CODE

Worked example: chi-square, by hand

WORKED EXAMPLE — The real sex × diagnosis table

	Arrhythmia	Normal
Male (observed):	112	82
Female (observed):	89	165

Step 1 — what would we EXPECT if sex and diagnosis were unrelated?

Expected counts, computed from the row/column totals: Male → 87.0 / 107.0 Female → 114.0 / 140.0

Step 2 — compare observed to expected, cell by cell.

Male-Arrhythmia: observed 112 vs expected 87 — a real, large gap in one direction.

Step 3 — chi-square adds up how big these gaps are, across all 4 cells:

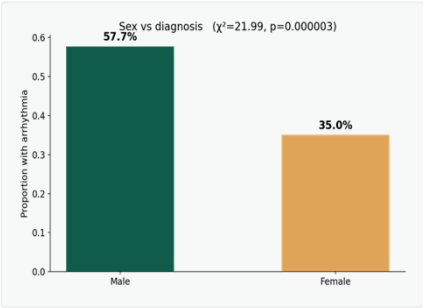
$\chi^2 = 21.99$, $p = 0.0000027$

The gaps between observed and expected are far too large to explain by chance alone — exactly why H0 gets rejected here.

A SIGNIFICANT FINDING

Sex vs diagnosis — the real result

- **Question:**
- is sex linked to diagnosis?
- **A real, visible gap between the two bars.**
- One sex shows a substantially higher arrhythmia rate than the other.
- **Result: reject H0.**
- p is far below 0.05 — sex and diagnosis are significantly associated in this sample.



COMPARING BOTH RESULTS

One test rejected, one didn't — on purpose

t-test: heart rate

Fail to reject H0

$p > 0.05$ — no significant mean difference detected.

Chi-square: sex

Reject H0

$p \ll 0.05$ — a real, significant association found.

The outcome depends only on the evidence in the data — never on which result you were hoping for.

RECAP

Key takeaways

- **Descriptive statistics come before charts, and charts come before tests.**
- Each step narrows down what's actually worth testing formally.
- **A chart is only as informative as the columns you feed it.**
- Height/weight and heart rate/QT interval showed real structure; most other pairs this week didn't — both are genuine findings.
- **Statistical rarity is not the same as being wrong.**
- Boxplot outliers get reviewed in context, not deleted on sight.
- **A non-significant result is not “proof of no effect.”**
- It means this test, on this sample, didn't find strong enough evidence — nothing more, nothing less.

69

RECAP

Key takeaways — Parts B & C

- **Descriptive statistics come before charts, and charts come before tests.**
- Each step narrows down what's actually worth testing formally.
- **Every formula in this deck was walked through by hand, with real numbers.**
- Skew, z-score, IQR, correlation, t-statistic, chi-square — not just code, the actual arithmetic underneath it.
- **Statistical rarity is not the same as being wrong.**
- Boxplot outliers get reviewed in context, not deleted on sight.
- **A non-significant result is not “proof of no effect.”**
- It means this test, on this sample, didn't find strong enough evidence — nothing more, nothing less.

70

Code Appendix

All executable code cells added in the same sequence as the Week 2 & Week 3 notes

Use them as a sequential notebook guide during lab support.
Each slide title matches the relevant section of the notes.
Long code blocks are split across consecutive appendix slides.

71

Code 01 — Import pandas and load the CSV

Sequential code appendix | added after original slide 100

```
import pandas as pd
df = pd.read_csv("arrhythmia.csv", header=None, na_values="?")
```

Appendix slide 1 of 53 | Source order from notes

72

Code 02 — Check the shape and look at the first rows

Sequential code appendix | added after original slide 100

```
print(df.shape)

print(df.head())
```


73

Code 03 — Name the first 15 columns, then check them

Sequential code appendix | added after original slide 100

```
df.columns = ["age","sex","height","weight","qrs_duration","pr_interval",
              "qt_interval","t_interval","p_interval","qrs_angle","t_angle",
              "p_angle","qrst_angle","j_angle","heart_rate"] \
             + list(df.columns[15:-1]) + ["class"]

print(df.columns[-3:])

print(df.head())
```


74

Code 04 — Keep an untouched copy of the data

Sequential code appendix | added after original slide 100

```
# Keep an untouched copy of the loaded and named dataset
df_original = df.copy(deep=True)
```


75

Code 05 — Keep an untouched copy of the data

Sequential code appendix | added after original slide 100

```
df_original stores the raw dataset.
```


76

Code 06 — Inspect the raw data before deciding what to clean

Sequential code appendix | added after original slide 100

```
# Check missing values
missing = df_original.isna().sum()
print("Columns containing missing values:")
print(missing[missing > 0].sort_values(ascending=False))

# Check duplicate rows
print("\nNumber of duplicate rows:")
print(df_original.duplicated().sum())

# Check the ranges of important measurements
range_cols = ["age", "height", "weight", "heart_rate"]

print("\nMinimum and maximum values:")
print(df_original[range_cols].agg(["min", "max"]))
```


77

Code 07 — Review age, height, and weight together

Sequential code appendix | added after original slide 100

```
review_cols = [
    "age",
    "sex",
    "height",
    "weight",
    "class"
]

records_to_review = df_original.loc[
    (df_original["age"] < 18) |
    (df_original["height"] < 120) |
    (df_original["height"] > 220) |
    (df_original["weight"] < 25),
    review_cols
].sort_values(
    ["age", "height", "weight"]
)

print(records_to_review.to_string())
```


78

Code 08 — Add BMI as a consistency check:

Sequential code appendix | added after original slide 100

```
records_to_review = records_to_review.copy()

records_to_review["bmi"] = (
    records_to_review["weight"] /
    (records_to_review["height"] / 100) ** 2
)

print(records_to_review.round(2).to_string())
```


79

Code 09.1 — Define the cleaning function

Sequential code appendix | added after original slide 100

```
def clean_data(data):
    # Work on a copy so the supplied DataFrame is not changed
    clean = data.copy(deep=True)

    # Standardise sex labels if letter labels are present
    clean["sex"] = clean["sex"].replace({
        "M": 0,
        "F": 1,
        "m": 0,
        "f": 1
    })

    # Calculate BMI to check whether height and weight
    # are internally consistent
    clean["bmi"] = (
        clean["weight"] /
        (clean["height"] / 100) ** 2
    )

    # Identify clearly unreliable demographic records
    invalid_record = (
        # A height above 250 cm is physically implausible
        (clean["height"] > 250)

        # A recorded height above 100 cm for age 0 or 1
        # is highly inconsistent with the recorded age
        | (
```


80

Code 09.2 — Define the cleaning function

Sequential code appendix | added after original slide 100

```
            (clean["age"] <= 1) &
            (clean["height"] > 100)
        )

        # An adult BMI below 10 suggests that the recorded
        # height and weight are internally inconsistent
        | (
            (clean["age"] >= 20) &
            (clean["bmi"] < 10)
        )
    )

    # Display the records before removing them
    print("Clearly unreliable records removed during cleaning:")

    print(
        clean.loc[
            invalid_record,
            [
                "age",
                "sex",
                "height",
                "weight",
                "bmi",
                "class"
            ]
        ].round(2)
```


81

Code 09.3 — Define the cleaning function

Sequential code appendix | added after original slide 100

```
    )

    # Remove the clearly unreliable records
    clean = clean.loc[~invalid_record].copy()

    # Recalculate BMI for the retained records
    clean["bmi"] = (
        clean["weight"] /
        (clean["height"] / 100) ** 2
    )

    # Fill ECG columns containing only a small
    # number of missing values
    for col in [
        "p_angle",
        "t_angle",
        "qrst_angle",
        "heart_rate"
    ]:
        clean[col] = clean[col].fillna(
            clean[col].median()
        )

    # Drop j_angle because most of its values are missing
    clean = clean.drop(
        columns=["j_angle"],
        errors="ignore"
```


82

Code 09.4 — Define the cleaning function

Sequential code appendix | added after original slide 100

```
    )

    # Remove exact duplicate rows
    clean = clean.drop_duplicates()

    # Create a simpler yes/no diagnosis column
    clean["arrhythmia_present"] = (
        clean["class"] != 1
    )

    # Reset the row numbers after records are removed
    clean = clean.reset_index(drop=True)

    return clean
```


83

Code 10 — Apply the cleaning function and review the result

Sequential code appendix | added after original slide 100

```
# Apply the cleaning function to the original data
df_clean_preview = clean_data(df_original)
```


84

Code 11 — Apply the cleaning function and review the result

Sequential code appendix | added after original slide 100

```
print("Original shape:", df_original.shape)
print("Cleaned shape:", df_clean_preview.shape)

print(
    "Rows removed:",
    len(df_original) - len(df_clean_preview)
)
```


85

Code 12 — Apply the cleaning function and review the result

Sequential code appendix | added after original slide 100

```
demographic_cols = [
    "age",
    "height",
    "weight",
    "bmi"
]

print("\nCleaned demographic ranges:")

print(
    df_clean_preview[demographic_cols]
    .agg(["min", "max"])
)
```


86

Code 13 — Apply the cleaning function and review the result

Sequential code appendix | added after original slide 100

```
print(
    "\nHeights above 250 cm:",
    (df_clean_preview["height"] > 250).sum()
)

print(
    "Infant records with height above 100 cm:",
    (
        (df_clean_preview["age"] <= 1) &
        (df_clean_preview["height"] > 100)
    ).sum()
)

print(
    "Adults with BMI below 10:",
    (
        (df_clean_preview["age"] >= 20) &
        (df_clean_preview["bmi"] < 10)
    ).sum()
)
```


87

Code 14 — Apply the cleaning function and review the result

Sequential code appendix | added after original slide 100

```
missing = df_clean_preview.isna().sum()

print("\nColumns still containing missing values:")

print(
    missing[missing > 0]
    .sort_values(ascending=False)
)

print(
    "\nDuplicate rows:",
    df_clean_preview.duplicated().sum()
)
```


88

Code 15 — Apply the cleaning function and review the result

Sequential code appendix | added after original slide 100

```
print(
    "\nj_angle still present:",
    "j_angle" in df_clean_preview.columns
)

print(
    "BMI column present:",
    "bmi" in df_clean_preview.columns
)

print(
    "arrhythmia_present column present:",
    "arrhythmia_present" in df_clean_preview.columns
)
```

Appendix slide 18 of 53 | Source order from notes

89

Code 16 — Apply the cleaning function and review the result

Sequential code appendix | added after original slide 100

```
print("\nFirst five cleaned records:")

print(
    df_clean_preview[
        [
            "age",
            "sex",
            "height",
            "weight",
            "bmi",
            "heart_rate",
            "class",
            "arrhythmia_present"
        ]
    ].head()
)
```

Appendix slide 19 of 53 | Source order from notes

90

Code 17 — Apply the cleaning function and review the result

Sequential code appendix | added after original slide 100

```
df_original should remain unchanged.
```

Appendix slide 20 of 53 | Source order from notes

91

Code 18 — Use SQLite

Sequential code appendix | added after original slide 100

```
import sqlite3
```

Appendix slide 21 of 53 | Source order from notes

92

Code 19 — Store the data using ETL and ELT

Sequential code appendix | added after original slide 100

```
# Use the cleaned data already reviewed above
df_etl_clean = df_clean_preview.copy(deep=True)

with sqlite3.connect("etl.db") as conn:
    df_etl_clean.to_sql(
        "clean",
        conn,
        if_exists="replace",
        index=False
    )

print("ETL completed.")
print("Cleaned table shape:", df_etl_clean.shape)
```


93

Code 20 — Store the data using ETL and ELT

Sequential code appendix | added after original slide 100

```
df_clean_preview has already been transformed.
```


94

Code 21.1 — Store the data using ETL and ELT

Sequential code appendix | added after original slide 100

```
with sqlite3.connect("elt.db") as conn:

    # Load the untouched raw data first
    df_original.to_sql(
        "raw",
        conn,
        if_exists="replace",
        index=False
    )

    # Read the raw table back into Pandas
    df_elt_raw = pd.read_sql(
        "SELECT * FROM raw",
        conn
    )

    # Transform the data after loading
    df_elt_clean = clean_data(df_elt_raw)

    # Store the cleaned result as a second table
    df_elt_clean.to_sql(
        "clean",
        conn,
        if_exists="replace",
        index=False
    )
```


95

Code 21.2 — Store the data using ETL and ELT

Sequential code appendix | added after original slide 100

```
print("ELT completed.")
print("Raw table shape:", df_elt_raw.shape)
print("Cleaned table shape:", df_elt_clean.shape)
```


96

Code 22 — Store the data using ETL and ELT

Sequential code appendix | added after original slide 100

```
df_original is loaded into the raw table first.
```


97

Code 23 — Confirm the stored tables

Sequential code appendix | added after original slide 100

```
with sqlite3.connect("etl.db") as conn:
    etl_tables = pd.read_sql(
        "SELECT name FROM sqlite_master WHERE type='table'",
        conn
    )

print("Tables in etl.db:")
print(etl_tables)

with sqlite3.connect("elt.db") as conn:
    elt_tables = pd.read_sql(
        "SELECT name FROM sqlite_master WHERE type='table'",
        conn
    )

print("\nTables in elt.db:")
print(elt_tables)
```


98

Code 24 — NumPy array, Pandas Series, Pandas DataFrame

Sequential code appendix | added after original slide 100

```
print(df_original["heart_rate"].mean())          # one number, a Series method

print(df_original[["heart_rate", "age"]].mean())    # one number PER column, a DataFrame
    method
```


99

Code 25 — Vectorisation and memory

Sequential code appendix | added after original slide 100

```
import time
# Slow method: loop
start = time.time()
slow_result = []
for i in range(len(df_original)):
    slow_result.append(
df_original["weight"].iloc[i] / df_original["height"].iloc[i]
    )
slow_time = time.time() - start
# Fast method: vectorised calculation
start = time.time()
fast_result = df_original["weight"] / df_original["height"]
fast_time = time.time() - start
# Compare the first five results
print("Slow result:")
print(slow_result[:5])
print("\nFast result:")
print(fast_result.head().tolist())
# Compare the speed
print("\nSlow time:", slow_time)
print("Fast time:", fast_time)
print("\nFast method is", slow_time / fast_time, "times faster")
```


100

Code 26 — Shape and data types

Sequential code appendix | added after original slide 100

```
print(df_original.shape)

print(df_original.dtypes.value_counts())

print(df_original.info())

print(df_original.describe())
```


101

Code 27 — Descriptive statistics

Sequential code appendix | added after original slide 100

```
summary_cols = ["age", "height", "weight", "heart_rate"]
print(df_etl_clean[summary_cols].describe())

skew_results = df_etl_clean[summary_cols].skew().round(2)
print(skew_results)

print(df_etl_clean["arrhythmia_present"].value_counts(normalize=True))

# Expected under the current rules: age and weight keep their raw minima.
print("Minimum age retained:", df_etl_clean["age"].min())
print("Minimum weight retained:", df_etl_clean["weight"].min())
```


102

Code 28 — Histograms: what they show

Sequential code appendix | added after original slide 100

```
import matplotlib.pyplot as plt

plt.hist(df_etl_clean["heart_rate"].dropna(), bins=20)
plt.xlabel("Heart rate"); plt.ylabel("Number of patients")
plt.title("Distribution of heart rate")
plt.show()
```


103

Code 29 — Histograms: what they show

Sequential code appendix | added after original slide 100

```
plt.hist(df_etl_clean["age"].dropna(), bins=25)
plt.xlabel("Age")
plt.ylabel("Number of patients")
plt.title("Distribution of age")
plt.show()
```


104

Code 30 — Histograms before cleaning

Sequential code appendix | added after original slide 100

```
import matplotlib.pyplot as plt

plt.hist(df_original["height"].dropna(), bins=30)
plt.title("Height (before cleaning)")
plt.xlabel("Height (cm)")
plt.ylabel("Number of patients")
plt.show()

plt.hist(df_original["weight"].dropna(), bins=30)
plt.title("Weight (before cleaning)")
plt.xlabel("Weight (kg)")
plt.ylabel("Number of patients")
plt.show()

plt.hist(df_original["heart_rate"].dropna(), bins=30)
plt.title("Heart rate (before cleaning)")
plt.xlabel("Heart rate (bpm)")
plt.ylabel("Number of patients")
plt.show()
```


105

Code 31 — Compare raw and cleaned demographic ranges

Sequential code appendix | added after original slide 100

```
# Compare the variables that were reviewed during cleaning
for col in ["age", "height", "weight", "heart_rate"]:
    print(
        col,
        "raw range:",
        (df_original[col].min(), df_original[col].max()),
        "clean range:",
        (df_etl_clean[col].min(), df_etl_clean[col].max())
    )
```


106

Code 32 — Compare raw and cleaned demographic ranges

Sequential code appendix | added after original slide 100

```
plt.hist(df_etl_clean["height"].dropna(), bins=30)
plt.title("Height (after cleaning)")
plt.xlabel("Height (cm)")
plt.ylabel("Number of patients")
plt.show()

plt.hist(df_etl_clean["weight"].dropna(), bins=30)
plt.title("Weight (after cleaning)")
plt.xlabel("Weight (kg)")
plt.ylabel("Number of patients")
plt.show()

plt.hist(df_etl_clean["heart_rate"].dropna(), bins=30)
plt.title("Heart rate")
plt.xlabel("Heart rate (bpm)")
plt.ylabel("Number of patients")
plt.show()
```


107

Code 33 — Boxplots: how to read them

Sequential code appendix | added after original slide 100

```
plt.boxplot(df_etl_clean["weight"].dropna(), vert=False, tick_labels=["Weight"])
plt.xlabel("Weight (kg)")
plt.show()          # Figure 10 is what you'll see
```


108

Code 34 — Step 1 – detect

Sequential code appendix | added after original slide 100

```
def iqr_flag(series):
    q1, q3 = series.quantile([0.25, 0.75])
    iqr = q3 - q1
    return (series < q1 - 1.5 * iqr) | (series > q3 + 1.5 * iqr)

age_flag = iqr_flag(df_etl_clean["age"])
height_flag = iqr_flag(df_etl_clean["height"])
weight_flag = iqr_flag(df_etl_clean["weight"])

demographic_flags = age_flag | height_flag | weight_flag
print(
    df_etl_clean.loc[
        demographic_flags,
        ["age", "sex", "height", "weight", "class"]
    ].sort_values(["age", "height", "weight"]).to_string()
)

# Heart rate is checked separately with a z-score
mean_hr = df_etl_clean["heart_rate"].mean()
std_hr = df_etl_clean["heart_rate"].std()
z_hr = (df_etl_clean["heart_rate"] - mean_hr) / std_hr
print("Heart-rate values with |z| > 3:")
print(df_etl_clean.loc[z_hr.abs() > 3, ["age", "sex", "heart_rate", "class"]])
```


109

Code 35 — Step 1 – detect

Sequential code appendix | added after original slide 100

```
print(
    df_etl_clean.loc[
        (df_etl_clean["age"] == 75) &
        (df_etl_clean["sex"] == 0) &
        (df_etl_clean["height"] == 190) &
        (df_etl_clean["weight"] == 80)
    ].to_string()
)
```


110

Code 36 — Step 2 – visualise

Sequential code appendix | added after original slide 100

```
fig, axes = plt.subplots(1, 3, figsize=(12, 4))
axes[0].boxplot(df_etl_clean["height"].dropna(), tick_labels=["Height"])
axes[0].set_title("Height"); axes[0].set_ylabel("cm")
axes[1].boxplot(df_etl_clean["heart_rate"].dropna(), tick_labels=["Heart rate"])
axes[1].set_title("Heart rate"); axes[1].set_ylabel("bpm")
axes[2].boxplot(df_etl_clean["weight"].dropna(), tick_labels=["Weight"])
axes[2].set_title("Weight"); axes[2].set_ylabel("kg")
plt.suptitle("Cleaned data before optional capping")
plt.show()
```


111

Code 37 — Step 3 - optional sensitivity check

Sequential code appendix | added after original slide 100

```
df_sensitivity = df_etl_clean.copy(deep=True)

df_sensitivity["height_capped"] = df_sensitivity["height"].clip(
    lower=df_sensitivity["height"].quantile(0.01)
)

df_sensitivity["heart_rate_capped"] = df_sensitivity["heart_rate"].clip(
    lower=df_sensitivity["heart_rate"].quantile(0.01),
    upper=df_sensitivity["heart_rate"].quantile(0.99)
)

df_sensitivity["weight_capped"] = df_sensitivity["weight"].clip(
    upper=df_sensitivity["weight"].quantile(0.99)
)
```


112

Code 38 — Step 4 – compare

Sequential code appendix | added after original slide 100

```
fig, axes = plt.subplots(1, 3, figsize=(12, 4))
axes[0].boxplot(df_sensitivity["height_capped"].dropna(), tick_labels=["Height"])
axes[0].set_title("Height (sensitivity-capped)"); axes[0].set_ylabel("cm")
axes[1].boxplot(df_sensitivity["heart_rate_capped"].dropna(), tick_labels=["Heart rate"])
axes[1].set_title("Heart rate (capped)"); axes[1].set_ylabel("bpm")
axes[2].boxplot(df_sensitivity["weight_capped"].dropna(), tick_labels=["Weight"])
axes[2].set_title("Weight (capped)"); axes[2].set_ylabel("kg")
plt.suptitle("Optional sensitivity-only capped view")
plt.show()
```

Appendix slide 42 of 53 | Source order from notes

113

Code 39 — Bar chart: arrhythmia rate by sex

Sequential code appendix | added after original slide 100

```
rate = df_etl_clean.groupby("sex", observed=False)["arrhythmia_present"].mean()
print(rate)

sex_labels = {0: "Male", 1: "Female"}
labels = [sex_labels[int(value)] for value in rate.index]

plt.bar(labels, rate.values)
plt.xlabel("Sex")
plt.ylabel("Proportion with arrhythmia")
plt.title("Arrhythmia rate by sex")
plt.show()          # Figure 13 is what you'll see
```

Appendix slide 43 of 53 | Source order from notes

114

Code 40 — Correlation heatmap

Sequential code appendix | added after original slide 100

```
import numpy as np

num_cols = ["age", "height", "weight", "qrs_duration", "pr_interval", "qt_interval",
    "heart_rate"]
corr_matrix = df_etl_clean[num_cols].corr()
print(corr_matrix.round(2))

# Find the strongest non-diagonal linear association
upper_triangle = corr_matrix.where(
    np.triu(np.ones(corr_matrix.shape), k=1).astype(bool)
)
strongest_pair = upper_triangle.abs().stack().idxmax()
print(
    "Strongest absolute correlation:",
    strongest_pair,
    round(corr_matrix.loc[strongest_pair[0], strongest_pair[1]], 2)
)

plt.imshow(corr_matrix, cmap="RdYlGn", vmin=-1, vmax=1)
plt.xticks(range(len(num_cols)), num_cols, rotation=45, ha="right")
plt.yticks(range(len(num_cols)), num_cols)
plt.colorbar(label="Correlation")
plt.title("Correlation between 7 of 279 input features")
plt.show()          # Figure 14 is what you'll see
```

Appendix slide 44 of 53 | Source order from notes

115

Code 41.1 — Pattern heatmap by arrhythmia type

Sequential code appendix | added after original slide 100

```
class_names = {
    1: "Normal", 2: "Ischemic changes (CAD)", 3: "Old Ant. MI", 4: "Old Inf. MI",
    5: "Sinus tachycardia", 6: "Sinus bradycardia", 7: "PVC", 8: "Supraventricular PC",
    9: "Left bundle branch block", 10: "Right bundle branch block",
    11: "1st degree AV block", 12: "2nd degree AV block", 13: "3rd degree AV block",
    14: "Left vent. hypertrophy", 15: "Atrial fib./flutter", 16: "Other",
}

pqrst_cols = ["p_interval", "qrs_duration", "pr_interval", "qt_interval", "t_interval",
    "heart_rate"]
class_counts = df_etl_clean["class"].value_counts().sort_index()
print("Patients per class:")
print(class_counts)

profile = df_etl_clean.groupby("class")[pqrst_cols].mean()
print("Mean profile per class:")
print(profile.round(1))

# standardise each column so differing scales (ms vs bpm) do not dominate the color map
profile_z = (profile - profile.mean()) / profile.std()
row_labels = [class_names[int(c)] for c in profile_z.index]

plt.imshow(profile_z.values, cmap="RdYlGn", vmin=-2, vmax=2, aspect="auto")
plt.xticks(range(len(pqrst_cols)), pqrst_cols, rotation=45, ha="right")
plt.yticks(range(len(profile_z)), row_labels)
plt.colorbar(label="Standardised mean (per column)")
plt.title("PQRST + heart rate profile, by arrhythmia type")
```

Appendix slide 45 of 53 | Source order from notes

116

Code 41.2 — Pattern heatmap by arrhythmia type

Sequential code appendix | added after original slide 100

```
plt.show()
```

Appendix slide 46 of 53 | Source order from notes

117

Code 42 — Missing-value bar chart

Sequential code appendix | added after original slide 100

```
miss = df_original.isna().mean().sort_values(ascending=False)
miss = miss[miss > 0] * 100
plt.barh(miss.index.astype(str), miss.values)
plt.xlabel("% missing")
plt.title("Missing values in the raw data")
plt.show()
```

Appendix slide 47 of 53 | Source order from notes

118

Code 43 — Scatter plot: heart rate by arrhythmia type

Sequential code appendix | added after original slide 100

```
order = sorted(df_etl_clean["class"].unique())
x_pos = df_etl_clean["class"].map({c: i for i, c in enumerate(order)})
plt.scatter(x_pos, df_etl_clean["heart_rate"], alpha=0.4)
means = df_etl_clean.groupby("class")["heart_rate"].mean()
print("Mean heart rate by class:")
print(means.sort_values())
plt.scatter(range(len(order)), [means[c] for c in order], color="red", marker="D",
    label="Mean per type")
plt.xticks(range(len(order)), [class_names[c] for c in order], rotation=45, ha="right")
plt.xlabel("Arrhythmia type"); plt.ylabel("Heart rate (bpm)")
plt.title("Heart rate by arrhythmia type")
plt.legend()
plt.show()
```

Appendix slide 48 of 53 | Source order from notes

119

Code 44 — Other scatter plots: useful and unhelpful comparisons

Sequential code appendix | added after original slide 100

```
colors = df_etl_clean["arrhythmia_present"].map({True: "orange", False: "teal"})
plt.scatter(df_etl_clean["height"], df_etl_clean["weight"], c=colors, alpha=0.5)
plt.xlabel("Height (cm)")
plt.ylabel("Weight (kg)")
plt.title("Height vs weight, by diagnosis")

from matplotlib.lines import Line2D
legend_handles = [
    Line2D([0], [0], marker="o", color="w", markerfacecolor="teal", markersize=8, label="No
        arrhythmia"),
    Line2D([0], [0], marker="o", color="w", markerfacecolor="orange", markersize=8,
        label="Arrhythmia present"),
]
plt.legend(handles=legend_handles)
plt.show()
```

Appendix slide 49 of 53 | Source order from notes

120

Code 45 — Other scatter plots: useful and unhelpful comparisons

Sequential code appendix | added after original slide 100

```
import numpy as np
jitter = df_etl_clean["arrhythmia_present"].astype(int) + np.random.uniform(-0.08, 0.08,
    len(df_etl_clean))
colors = df_etl_clean["arrhythmia_present"].map({True: "orange", False: "teal"})
plt.scatter(df_etl_clean["age"], jitter, c=colors, alpha=0.5)
plt.yticks([0, 1], ["No", "Yes"])
plt.xlabel("Age")
plt.ylabel("arrhythmia_present")
plt.title("Age vs arrhythmia_present")
plt.show()          # y-axis labels (No/Yes) already identify the two colours here
```


121

Code 46 — Pair plot: scan several relationships at once

Sequential code appendix | added after original slide 100

```
colors = df_etl_clean["arrhythmia_present"].map({True: "orange", False: "teal"})
cols = ["height", "weight", "qt_interval", "heart_rate"]
fig, axes = plt.subplots(4, 4, figsize=(9, 9))
for i, c1 in enumerate(cols):
    for j, c2 in enumerate(cols):
        ax = axes[i, j]
        if i == j:
            ax.hist(df_etl_clean[c1].dropna(), bins=20)
        else:
            ax.scatter(df_etl_clean[c2], df_etl_clean[c1], s=4, alpha=0.35, c=colors)
        if i == 3:
            ax.set_xlabel(c2)
        if j == 0:
            ax.set_ylabel(c1)

plt.tight_layout()

from matplotlib.patches import Patch
fig.legend(handles=[
    Patch(color="teal", label="No arrhythmia"),
    Patch(color="orange", label="Arrhythmia present"),
], loc="upper right")
plt.show()
```


122

Code 47 — t-test: heart rate by diagnosis

Sequential code appendix | added after original slide 100

```
%pip install scipy
from scipy import stats

a = df_etl_clean.loc[df_etl_clean["arrhythmia_present"], "heart_rate"]
b = df_etl_clean.loc[~df_etl_clean["arrhythmia_present"], "heart_rate"]

t, p = stats.ttest_ind(a, b, equal_var=False)
print("Mean with arrhythmia:", a.mean())
print("Mean without arrhythmia:", b.mean())
print(f"t = {t:.2f}, p = {p:.4f}")

if p < 0.05:
    print("Reject H0: the mean heart rates differ significantly.")
else:
    print("Fail to reject H0: no significant mean difference was detected.")
```


123

Code 48 — Chi-square: sex vs diagnosis

Sequential code appendix | added after original slide 100

```
table = pd.crosstab(
    df_etl_clean["sex"],
    df_etl_clean["arrhythmia_present"]
)
print(table)

chi2, p, dof, expected = stats.chi2_contingency(table)
print(f"chi2 = {chi2:.2f}, p = {p:.6f}")

if p < 0.05:
    print("Reject H0: sex and diagnosis are associated in this sample.")
else:
    print("Fail to reject H0: no significant association was detected.")
```


124